
Project: Unitree G1 Soccer Skills

CS2810 Teaching Team

1 Overview

This is the project component of Embodied AI CS2810 (Spring 2026). The task is to develop reinforcement learning skills for humanoid robot soccer using the Unitree G1 (29 dof) platform in mjlab physics simulation. The project centers on two tasks:

- **Shooting:** The G1 robot must approach and kick a stationary ball toward a goal. This is a perception-guided motor skill requiring motion tracking priors and ball trajectory prediction.
- **Goalkeeping:** The G1 robot must intercept an incoming ball launched along a parabolic trajectory. This is a reactive interception task requiring rapid end-effector positioning and ball trajectory estimation.

2 Project Phases & Grading Rubric

The project is evaluated out of 100 total points, broken down into three primary components: Phase 1 (60%), Phase 2 (30% = 15% Shooting + 15% Goalkeeping), and Submission Deliverables (10%).

2.1 Phase 1: Basic Shooting and Goalkeeping (60% of project score)

Implement single-agent training for individual shooter and goalkeeper tasks in a non-adversarial setting. You may refer to [1] and [2] for guidance on environment design, reward shaping, and training strategies. This phase evaluates the fundamental motor control and perception integration of your policies.

The 60% allocation is divided equally between the two tasks (30% each), calculated over 50 evaluation trials:

- **Shooter Policy (30 points total):** Evaluated using `eval_naive_shooter.py`. Success is defined as the ball completely crossing the goal plane inside the frame posts.
- **Goalkeeper Policy (30 points total):** Evaluated using `eval_naive_goalkeeper.py`. Success is defined as preventing the ball from entering the goal (ball does not cross the goal plane inside the frame).

Both tasks are scored with a linear formula applied to the success/block rate over 50 evaluation trials:

$$\text{score} = \max\left(0, \frac{\text{success_rate} - 0.8}{0.2}\right) \times 30$$

Success rates below 80% receive 0 points. At 80% the score is 0, at 100% the score is 30, with linear interpolation in between.

Reproducibility. The eval scripts (`eval_naive_shooter.py` and `eval_naive_goalkeeper.py`) support a `--seed` flag (default: 2810) that seeds Python, NumPy, PyTorch, CUDA, and the environment RNGs, making each run fully reproducible. For your Phase 1 report, you must evaluate your policies with **three different seeds**: 42, 2810, and 202606. Run 50 trials per seed, compute the mean success/block rate across the three runs, and apply the scoring formula above using this averaged rate. Reporting the mean over three independent seeds provides a more stable and fair metric than a single 50-trial run.

2.2 Phase 2: Cross-Evaluation Tournament (30% of project score)

Phase 2 is an all-to-all cross-evaluation tournament conducted centrally by the teaching team. The 30% allocation is divided equally between shooting and goalkeeping (15% each). After completing basic single-agent policies in Phase 1, teams continue to improve their agents—through adversarial training, self-play, or further single-agent optimization—and then compete against every other team.

Because each team will design unique observation spaces, feature extractors, and neural network architectures, your codebases will be heterogeneous. To enable cross-evaluation without requiring teams to share code or checkpoints, the tournament adopts a standardized API interface: each team deploys their trained policy behind a lightweight server and submits the server URLs to the teaching team's web console. The web console then launches the match executor, which loads two G1 robots into a single MuJoCo scene and queries each team's server at every simulation step.

2.2.1 API Protocol

All teams must implement a policy server exposing the following two endpoints (a reference implementation is provided as `phase2/api_server.py`):

- `POST /act` — Receives raw MuJoCo state for both robots and the ball, and returns an action for the robot the server controls (determined by the `--task` flag).

```
Request: {
  "shooter": {root_pos, root_quat, root_lin_vel, root_ang_vel,
              joint_pos, joint_vel, last_action},
  "goalkeeper": {...},
  "ball": {pos, vel}
}
Response: {"action": [[f32, ...]]}
```

- `POST /reset` — Resets the policy's internal state (LSTM hidden state, history buffer, etc.).

```
Request: {}
Response: {"status": "ok"}
```

The tournament server sends the same raw state to both servers regardless of their role—each server extracts its own robot's data to compute observations.

2.2.2 Timeline and Milestones

- **Weeks 13–16: Agent Development & Improvement**

Teams first complete their basic single-agent shooting and goalkeeping policies (Phase 1). They then continuously improve their agents' capabilities through adversarial training, self-play, or further single-agent optimization. And this phase will continue until 6.25.

- **Week 17: Report Submission & Cross Evaluation**

The final report and all required submission materials are due by June 26. By the same deadline, each team must deploy its policy servers (which must be network-reachable from the tournament server) and submit the access URLs. Cross evaluations will begin on June 26 and will be conducted centrally on a server provided by the teaching team. The teaching team uses the `phase2/tournament_server.py` web console to start and monitor matchups. For each pair of teams (Team A and Team B), both directions are evaluated:

- Team A's Shooter vs. Team B's Goalkeeper (10 trials)
- Team B's Shooter vs. Team A's Goalkeeper (10 trials)

The specific evaluation schedule will be announced before June 26.

2.2.3 Scoring Mechanism

There are 14 teams in total. Each team's Phase 2 score is computed separately for shooting and goalkeeping (15 points maximum each):

- **Base Score:** Every team starts with 2 points for each task.

- **Head-to-Head Points:** For each opponent team, a direct matchup is played. Taking Team A's Shooter vs. Team B's Goalkeeper as an example: 10 trials are run. If Team A's shooter scores > 5 goals, Team A earns +1 point; otherwise (i.e., the goalkeeper holds the shooter to ≤ 5 goals), Team B earns +1 point. The same rule applies symmetrically to Team A's Goalkeeper vs. Team B's Shooter.

Each team plays every other team once per task direction, yielding 13 head-to-head opportunities per task. The maximum score per task is therefore $2 + 13 = 15$ points.

2.3 Submission and Deliverables (10% of project score)

Submission consists of a PDF report and the corresponding code repository before the end of Week 17 (6.26) to gradescope. The PDF report must include:

- **Phase 1 Results:** Basic shooting and goalkeeping evaluation results (success/block rates over 50 trials) with analysis.
- **Phase 2 Design & Approach:** Description of how you improved your agents for the tournament (adversarial training, self-play, or further single-agent optimization). Explain your API server implementation, including how you compute observations from raw state. Since the tournament is conducted centrally by the teaching team, you do not need to report cross-evaluation results yourself.

3 Using the Template

The template is given at <https://github.com/xiaojxkevin/G1-mjlab-soccer>.

3.1 Repository Structure

The template provides:

- **Playable environments:** Unitree-G1-Shooter and Unitree-G1-Goalkeeper registered as mjlab tasks with configurable scene layout, ball physics, and MuJoCo visualization.
- **Evaluation scripts:** `eval_naive_shooter.py` and `eval_naive_goalkeeper.py` that load trained checkpoints, run headless batch trials, collect paper-matching metrics, and record videos. And you are allowed to modify these scripts if you want to use different observation spaces, reward functions etc.
- **Training configs:** Training environment designs under `config/training/` that specify observation spaces, reward functions, termination conditions, and domain randomization from the two papers. And it should be noticed that these are generated with coding agents and may contain errors. So you should carefully review and debug them before use.

3.2 Play and Visualization

Use the `scripts/play.py` script to visualize the environment:

```
# Shooter
python scripts/play.py Unitree-G1-Shooter --agent zero --viewer native

# Goalkeeper
python scripts/play.py Unitree-G1-Goalkeeper --agent zero --viewer native
```

3.3 Evaluation

Use the eval scripts to benchmark trained policies. See Section 1.1 for the required seed and trial settings for Phase 1 scoring.

```
# Shooter
python scripts/eval_naive_shooter.py --headless --num-trials 50 \
```

```
--checkpoint <path>

# Goalkeeper
python scripts/eval_naive_goalkeeper.py --headless --num-trials 50 \
    --checkpoint <path>
```

3.4 Implementing Training

The `train.py` script in `scripts/` serves as the training entrypoint. It is expected to register their own training tasks, implement the model classes, and run PPO training.

3.5 Cross-Evaluation Competition

For Phase 2 cross-evaluation, teams deploy their policies as API servers and submit their API URLs through the teaching team’s web console. The direct match executor `phase2/compete.py` is mainly a teaching-side implementation detail and debugging entrypoint; the expected workflow for students is to keep their API servers running and provide URLs that are reachable from the tournament server.

```
# Start policy servers (one per robot)
python phase2/api_server.py --checkpoint <path> --port 8000 --task shooter
python phase2/api_server.py --checkpoint <path> --port 8001 --task goalkeeper
```

In the web console, the teaching team enters the shooter team name, shooter API URL, goalkeeper team name, goalkeeper API URL, and number of trials. The console displays a Viser URL for each running match; after opening that page, the match begins only after clicking **Start Trials**. The server then loads two G1 robots into a single MuJoCo scene, reads raw MuJoCo state (joint angles, root poses, ball position/velocity), and sends the same state dict to both teams’ API servers. Each team’s server computes its own observation tensor and returns an action. Each episode lasts 5 seconds. This design fully decouples observation spaces across teams. A reference API server with example observation computation is provided as `phase2/api_server.py`. More detailed student-side instructions are provided in `phase2/README_CLIENT.md`.

3.6 Contributing and Reporting Issues

This template is a work in progress. The training configs, environment wrappers, and evaluation scripts were partially generated by coding agents and may contain bugs or rough edges. **We strongly encourage all students to:**

- **Report issues** — if you find a bug, a broken config, or unclear documentation, open a GitHub Issue on the template repository.
- **Submit pull requests** — fixes, improvements, and additional utilities (e.g., better observation configs, visualization tools, or multi-agent helpers) are welcome. PRs that benefit the whole class will be merged and acknowledged.

Treat this repository as a living codebase that improves with your feedback throughout the semester.

References

- [1] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, “Learning soccer skills for humanoid robots: A progressive perception-action framework,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.05310>
- [2] J. Ren, J. Long, T. Huang, H. Wang, Z. Wang, F. Jia, W. Zhang, J. Wang, P. Luo, and J. Pang, “Humanoid goalkeeper: Learning from position conditioned task-motion constraints,” 2025.